# Project 4: Dynamic Routing Mechanism Design in Faulty Network Using Dijkstra's Algorithm

Instructor: Tapadhir Das
Group: Tristan Ramirez, Minh Huy Tran, Rei Thao

# Table of Contents

**I. Functionality of the Protocol** ..... **3**
I.a) Overview ..... 3
I.b) Protocol Functionality ..... 3
**II. Novel Contribution** ..... **3**
II.a) Contributions ..... 3
II.b) Visual Simulation ..... 4
II.c) Novelty Justification ..... 4
**III. Results and Analysis** ..... **4**
III.a) Experimental Setup ..... 4
III.b) Sample Results ..... 5
III.c) Weighted Links ..... 8
III.d) Analysis ..... 9
**VI. Conclusion** ..... **9**

# I. Functionality of the Protocol

## I.a) Overview

The protocol provides dynamic routing in a mesh network where nodes and links may fail, disconnect, or recover. The program accounts for route reconfiguration, failure handling, and recovery by implementing Dijkstra's algorithm to calculate routes to establish the most efficient connection between a start and goal node and find alternative connections under random failure conditions. The model and simulation are able to fully imitate an adaptive, fault tolerant mesh network topology.

## I.b) Protocol Functionality

The Network class maintains graph structure, failed nodes, and failed links. The neighbor-selection logic excludes failed components, mirroring real link-state routing behavior. Failures are introduced with a 5% probability at each timestep, over ten timesteps, with independent node and link failure chances. Routing is performed using Dijkstra's algorithm. When the destination or needed intermediate nodes are failed, the algorithm will not return an available path. Recovered nodes and links will rejoin the routing graph in following timesteps.

The Dijkstra's algorithm implemented has the time complexity of $O((N+E)\log N)$, close to $O(n \log n)$, which is one of the most efficient path finding algorithms implementable on the known network topography, making the program function in a timely manner with large-scale topology.

## I.c) Real-life Application and Simulation

This protocol is widely used on networks that prioritize latency and fault tolerance, some examples that prove this protocol to be most effective are IP routing, network design and planning, Quality of Service routing, and network congestion control. Since the algorithm dynamically adapts to changes in nodes and links, it provides benefits to be implemented in wireless networks such as WiFi and telecommunications, which are notorious for having volatile topologies due to alternative connection medium. The protocol's ability to detect link weights provide its practicality to non-network applications such as GPS, where the current location, destination, and intersections are nodes and streets are links, AI reinforcement learning, social networking, or DNA sequencing.

The simulation provided is applicable to all the aforementioned real-life applications with the concept being simplified into nodes and links being volatile and dynamically altered in an adaptive topology.

# II. Novel Contribution

## II.a) Contributions

An extension introduces risk-aware routing, where routes are evaluated not only by hop cost but also by likelihood of failure. Paths with lower risk scores are preferred, improving reliability compared to traditional shortest-path routing.

Additionally, users are able to choose between a custom, user-defined network or static, built-in network. If the pre-defined network is desired, a 9-node mesh network shall be used for the simulation. This allows for experimentation and repeatability.

Users are able to simulate both manual failure/repair and random failure simulations. Under manual failure/repair mode, nodes and links will be manually repaired or failed. This mode is ideal for debugging and simulating controlled environments. Random failure simulation mode randomly fails nodes or links each time step with a 5% failure chance per node/link.

## II.b) Visual Simulation

```
Start node: N1
Goal node: N9
Best path from N1 to N9 : ['N1', 'N2', 'N3', 'N6', 'N9']

=== TIME STEP 0 ===

Failed Nodes: []
Failed Links: []
Route from N1 to N9: ['N1', 'N2', 'N3', 'N6', 'N9']

=== TIME STEP 1 ===

Failed Nodes: ['N3', 'N5']
Failed Links: [('N5', 'N4')]
Route from N1 to N9: ['N1', 'N4', 'N7', 'N8', 'N9']

=== TIME STEP 2 ===

Failed Nodes: []
Failed Links: []
Route from N1 to N9: ['N1', 'N2', 'N3', 'N6', 'N9']

=== TIME STEP 3 ===
```

## II.c) Novelty Justification

Unlike reactive routing that recalculates paths after failure, risk-aware routing is prepared for instability by proactively avoiding high-risk paths through a combinationed evaluation of cost and failure probability. This modification improves reliability in unstable environments by greatly reducing the probability of disconnection/failure. When

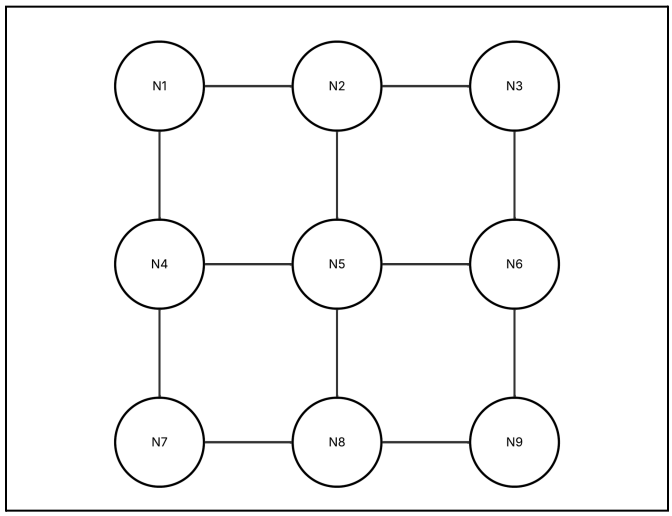
combined with adaptive recovery methods, connectivity is strengthened in dynamic and unpredictable environments.

By offering both manual and random failure modes, the program provides a method to study fault tolerance in both controlled and random environments. . This approach emphasizes the difference between reactive routing and aware routing, allowing users to learn, teach, and research the tool and routing mechanisms.

# III. Results and Analysis

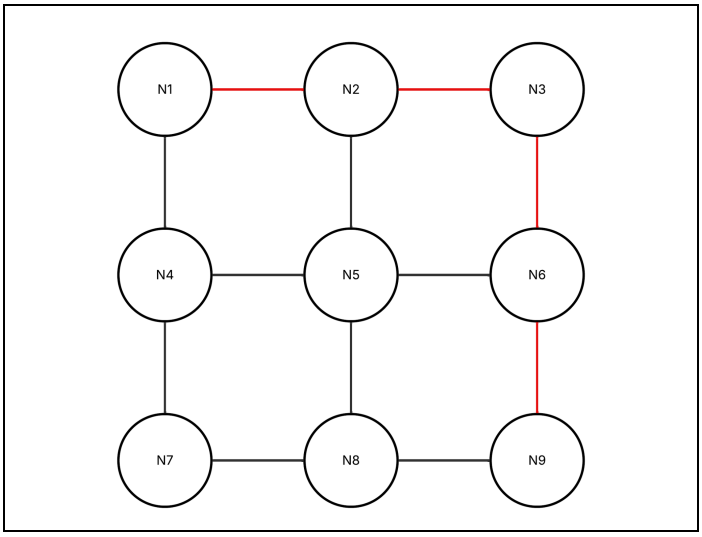
## III.a) Experimental Setup

A 9-node mesh network was simulated for 10 timesteps with node and link failure probabilities of 0.05. The routing objective was connecting N1 to N9.



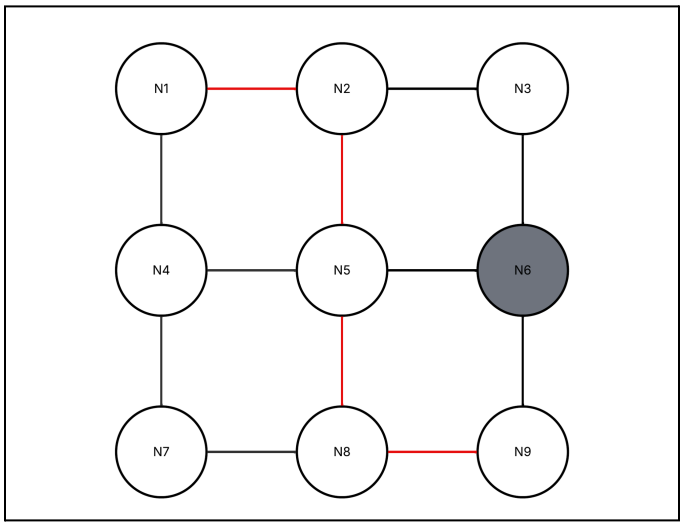

## III.b) Sample Results

- **Timestep 0:** Normal operation, route found.
  Failed Nodes: []
  Failed Links: []
  Route from N1 to N9: ['N1', 'N2', 'N3', 'N6', 'N9']




This demonstrates that the program is able to use Dijkstra's algorithm to find the fastest path from N1 to N9. Naming is also a hidden rule in this algorithm because the program checks the nodes alphabetically, hence N2 is chosen instead of N4, which technically is correct. However, since the objective of this program is to find the most efficient path, the ability to find one is sufficient. Since all links have the same weight of 1, the most efficient path will traverse through 4 links and 3 nodes to get to the destination.

- **Timestep 1:** Node N6 failed, rerouting successful.
  Failed Nodes: ['N6']
  Failed Links: []
  Route from N1 to N9: ['N1', 'N2', 'N5', 'N8', 'N9']

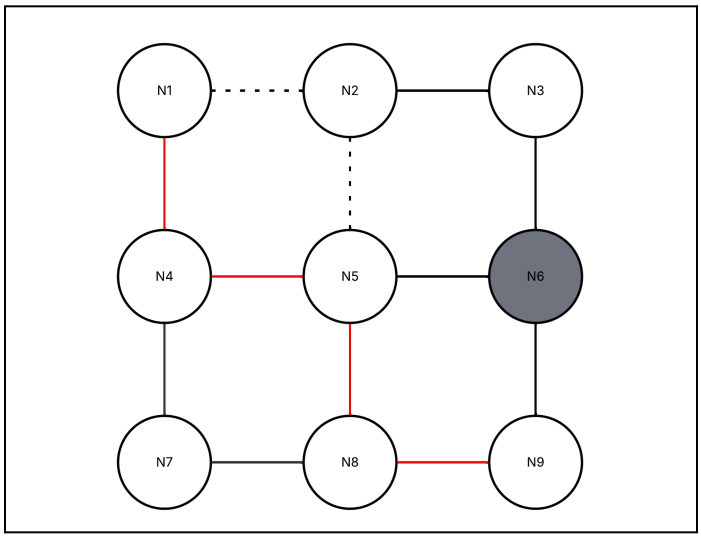



This demonstrates the program's ability to reroute to the next most efficient path if nodes on the current path are down. Node N6 was originally part of the found most efficient path, since the program detects error, it moves on to the path with similar

efficiency. This can be replicated on every node given that there is a viable path from N1 to N9.

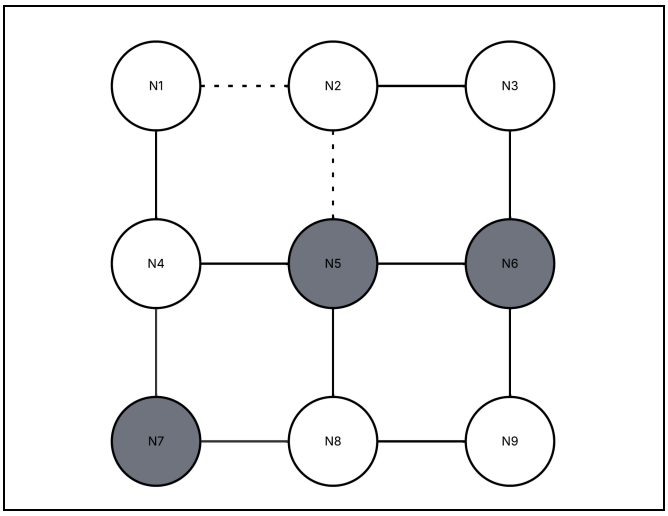
The program reroutes every time there is a change in the topology. If node N6 is to be reconnected, even though the current path is the most efficient, the program still shows the path at Timestep 0, which has the same efficiency as the previous path.

- **Timestep 2:** Link (N1,N2) failed, alternative path located.
  Failed Nodes: ['N6']
  Failed Links: [('N1', 'N2')]
  Route from N1 to N9: ['N1', 'N4', 'N5', 'N8', 'N9']



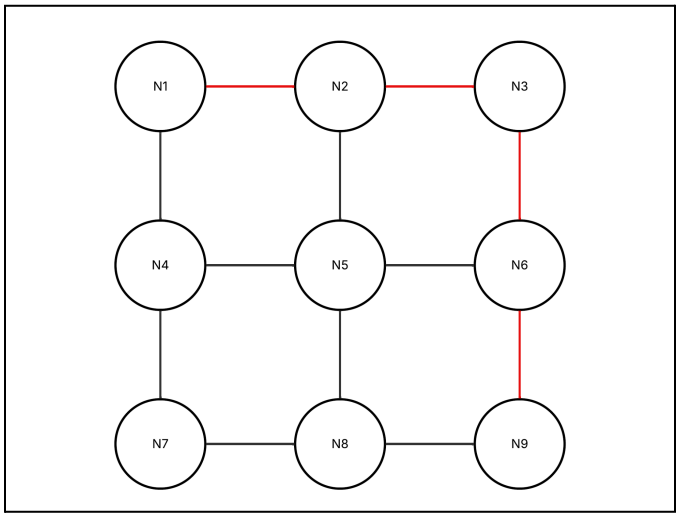

  Similarly, if links are disconnected, the program is able to find an alternative path.

- **Timestep 3**: Nodes N5 and N7 failed, no path available.
  Failed Nodes: ['N6', 'N5', 'N7']
  Failed Links: [('N1', 'N2')]
  No available path (network disconnected)




If no viable path is found, the program displays a disconnected network but still maintains the topology if a node or link is to be repaired and re-establish a path. The program checks every viable node to move forward, in this scenario, the node is N4. Since both nodes connected to N4 are inaccessible, the program recognizes that there is no viable path to get to the goal node.

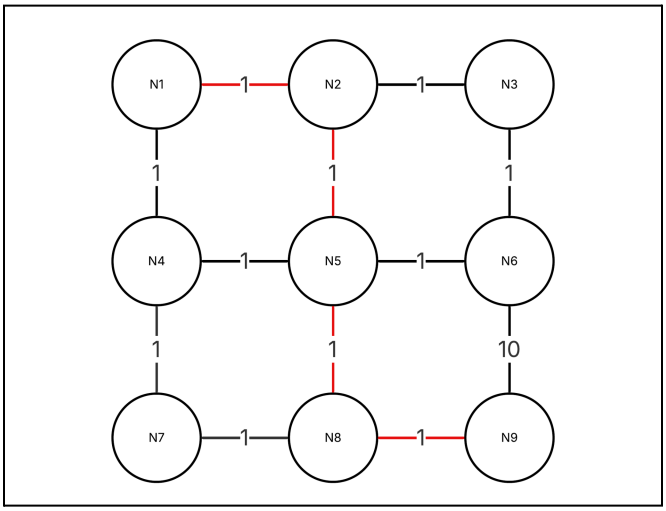
- **Timestep 4:** Network healed, routing restored.
  Failed Nodes: []
  Failed Links: []
  Route from N1 to N9: ['N1', 'N2', 'N3', 'N6', 'N9']



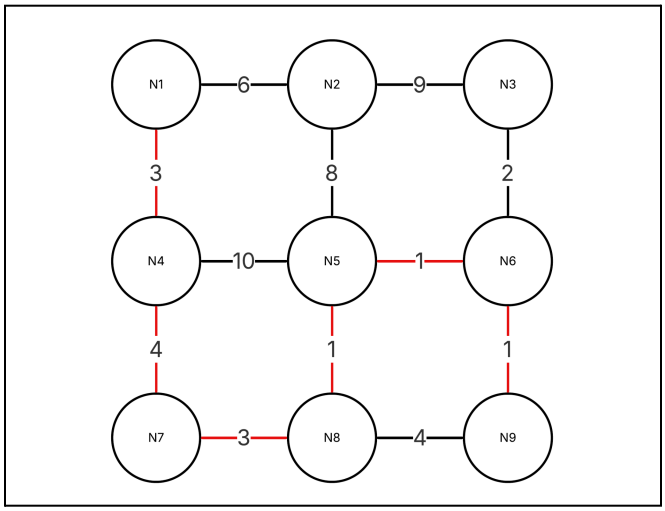

The program is able to keep track of the topology and is able to find a route when nodes and links are fixed.

## III.c) Weighted Links

Given the default topology but increasing the weight of the link N6-N9 forces the algorithm to find a more efficient path. The program recalculates the optimal path and takes into consideration the weight of the links between nodes. The variance in link weights can be attributed to real-life factors such as latency, throughput, distance, and connection medium quality.




In order to make the simulation more realistic, multiple weighted links are also taken into account to find the most efficient path.




## III.d) Analysis

The simulated mesh network provides a controlled environment to study how Dijkstra's algorithm behaves under dynamic failure conditions. The grid-like structure ensures that most nodes have alternative paths toward the destination, giving the program multiple failure events before becoming disconnected to maximize fault tolerance. Intermediate nodes serve as high-degree hubs whose failures have disproportionately

large impact relative to the failure of edge nodes. This shows that more interconnected nodes exhibit high efficiency but also high vulnerability when they fail. A major strength of the program is its adaptability upon recovery. Once failed nodes or links are repaired, they are immediately reinserted into the routing algorithm. The program is able to find the shortest path again, even if that path differs from the previously used one. Implementing varying link weights demonstrates that the algorithm correctly prefers lower-cost paths, even when those alternatives include more hops between nodes.

With the time complexity being O((N +E)logN), the system scales efficiently with network size. Although the algorithmic efficiency consideration is negligible in a 9-node mesh topology, this remains practical even for thousands of nodes. Given that routing protocols in real networks recompute frequently, the use of Dijkstra is justified both theoretically and by practice.

# VI. Conclusion

This project displays the value of implementing Dijkstra's Algorithm in dynamic routing involving node/link failures in mesh networks. By simulating both manual and random failure scenarios, the program shows how routes can be recalculated in real time to maintain connectivity, while repaired nodes and links are able to reintegrate into the topology. The introduction of risk-aware routing further improves versatility by proactively avoiding unstable paths and reducing the likelihood of disconnection, ensuring stronger reliability in volatile environments. Together, these contributions highlight a fault-tolerant framework that is able to demonstrate efficiency, adaptability, and reliability, making it valuable for both practical applications and educational research of resilient network routing mechanisms.